N O O T E R R A

How It Works

A live model of your business that watches everything, predicts what happens next, and acts on your behalf only as fast as you trust it.

Confidential. For qualified partners and investors.
April 2026

Contents

1. What Nooterra Is . . . 3
   1.1. What the user sees . . . 4
2. The Problem: Why AI Agents Fail in Business . . . 5
   2.1. The cost is real . . . 5
3. How It Works . . . 7
   3.1. Stage 1: Observe . . . 7
   3.2. Stage 2: Model . . . 7
   3.3. Stage 3: Predict . . . 7
   3.4. Stage 4: Act . . . 8
   3.5. Stage 5: Evaluate . . . 8
   3.6. Stage 6: Learn . . . 8
4. What Is Built Today . . . 9
   4.1. The eight dashboard views . . . 9
5. The World Model: How Predictions Work . . . 10
   5.1. From events to objects . . . 10
   5.2. From objects to predictions . . . 10
   5.3. What confidence actually means . . . 10
   5.4. The role of the AI model . . . 11
6. Earned Autonomy: How Trust Works . . . 12
   6.1. The five trust levels . . . 12
   6.2. How promotion works . . . 12
   6.3. How demotion works . . . 12
   6.4. Why this matters for adoption . . . 12
7. Why Nooterra Wins . . . 13
   7.1. The competitive landscape . . . 13
   7.2. What about the big AI companies? . . . 13
   7.3. The four moats . . . 13
8. Commercial Model . . . 15
   8.1. Unit economics . . . 15
   8.2. Go-to-market . . . 15
9. The Vision: From Wedge to Platform . . . 16

1. What Nooterra Is

Every AI company right now is building agents that can do things. Send emails. Update records. File tickets. Move data between systems.

None of them can tell you what happens next.

A collection email sent to the wrong customer at the wrong time can trigger a disputed charge, a churned account, and a missed renewal. A refund processed without checking the support ticket history can undermine a relationship your team spent months building. A payment reminder sent while a retry is already scheduled makes your company look incompetent.

Human employees avoid these mistakes because they carry a mental model of the business. They know the customer, the context, the history, and the likely consequences of their actions. AI agents do not.

Nooterra fixes this. It is an enterprise world model platform: a system that maintains a live, continuously updated representation of your company's operational state. It watches every event across your connected business systems, builds a map of every customer, invoice, payment, and conversation, predicts what happens next, and acts on your behalf through a governed process where trust is earned from evidence.

The system runs in a continuous loop:

THE NOOTERRA LOOP

1. **Observe** — Watch Stripe, Gmail, QuickBooks, CRM in real time. Every invoice, payment, email, and event is captured.
2. **Model** — Build a live map of every customer, invoice, payment, and conversation. Track relationships across systems.
3. **Predict** — Estimate hidden state. "72% chance this invoice gets paid in 7 days." "This customer's churn risk just jumped."
4. **Act** — Propose the right action with evidence. Send the right email, to the right person, at the right time.
5. **Evaluate** — Grade every action. Did it follow the rules? Did it achieve the goal? Did anything go wrong?
6. **Learn** — Model gets more accurate. Agent earns more autonomy. The system improves from its own operations.

This loop runs continuously. Every event updates the model. Every action generates evidence. Every outcome improves the predictions. The system gets better the longer it runs.

1.1. What the user sees

The experience is simple:

1. Connect Stripe (takes 2 minutes)
2. See your company state: every customer, invoice, and payment relationship mapped automatically
3. Get proposals: “Send a soft reminder to Acme Corp about Invoice #4821. Here’s why.”
4. Approve or reject with one click
5. Watch autonomy expand: proven actions start running automatically. A single mistake triggers immediate human review.

The complexity is behind the surface. The operator sees proposals and outcomes. The system handles observation, modeling, prediction, policy evaluation, and trust management.

2. The Problem: Why AI Agents Fail in Business

Most AI systems deployed in business today follow one of three patterns.

Assistants answer questions and draft content. They are useful, but passive. They do not take action.

Automations move data between systems on predefined triggers. They are reliable for simple rules ("if invoice overdue, send template email") but break when context matters. A templated reminder sent to a customer with an open support ticket and a pending renewal is worse than no reminder at all.

Agents are given tools and goals, then allowed to act. They are powerful. They are also dangerous when they lack a coherent picture of the system they operate inside.

All three patterns share the same structural weakness. They are thin on state. They see the task in front of them. They do not see the web of relationships, constraints, and consequences around it.

FIGURE 1: WHAT AGENTS MISS

What the agent sees:

Invoice #4821 from Acme Corp
Overdue 14 days, $4,200
Available action: send collection email

→

What actually happens:

Stripe: payment retry already scheduled.
Customer receives two conflicting messages.

Support: open ticket escalated to manager.
Collection email undermines the relationship.

CRM: contract renewal in 9 days.
Churn probability jumps from 40% to 78%.

This problem has a name. Academic researchers call it **dynamics blindness**: the inability of AI agents to predict the hidden state transitions their actions cause. A benchmark study published in January 2026 ("World of Workflows," arXiv) tested frontier AI models against 234 enterprise tasks with over 4,000 active business rules. The best models consistently failed to predict which rules would fire, what side effects would propagate, and which constraints would break.

The conclusion: enterprise AI needs more than better prompts or more reasoning power. It needs an explicit model of business state and business dynamics.

That model is what Nooterra builds.

2.1. The cost is real

U.S. small businesses with unpaid invoices carry $17,500 in overdue receivables on average (QuickBooks, May 2025). Mid-market firms carry approximately $304,000. Late payments regularly create payroll stress for SMBs (Bluevine, March 2026). These are not technology problems. They are dynamics problems: businesses lack the operational intelligence to predict which invoices

will go overdue, which customers are at risk, and what actions will accelerate collection without damaging relationships.

3. How It Works

The Nooterra loop has six stages. Each one builds on the previous. Here is what happens at each stage, what the user sees, and what the system does internally.

3.1. Stage 1: Observe

What happens: The system connects to your business tools and watches everything in real time. When an invoice is created in Stripe, a payment fails, a dispute is opened, or a customer email arrives, the system captures it immediately.

What the user sees: An activity feed showing every event as it happens. “Invoice #4821 created for Acme Corp, $4,200, due March 28.”

What the system does: Each event is typed, timestamped, and recorded in an append-only event ledger with hash-chain integrity (the same tamper-evidence technique used in financial audit systems). Every event carries provenance: which system it came from, how it was extracted, and how confident the extraction is.

3.2. Stage 2: Model

What happens: Events are transformed into a structured map of your business. Customers, invoices, payments, disputes, and conversations become objects with relationships between them. “Acme Corp owes Invoice #4821. Invoice #4821 has 3 related support tickets. Acme Corp’s contract renews in 9 days.”

What the user sees: A company state view showing every entity and its relationships. Click on a customer and see their invoices, payments, communications, and support history in one place.

What the system does: Raw events are transformed into canonical objects in a versioned graph. The system resolves entities across sources (the “Acme Corp” in Stripe is the same as “ACME Corporation” in QuickBooks). Every object carries version history, so the system can reconstruct what it knew about any entity at any point in time.

3.3. Stage 3: Predict

What happens: The system estimates things it cannot directly observe. “This invoice has a 34% chance of being paid in the next 7 days.” “This customer’s churn risk is 71%.” “There is a 22% probability of a dispute on this charge.”

What the user sees: A prediction dashboard showing risk scores, payment probabilities, and cash flow forecasts. Each prediction shows its confidence level and the model’s historical accuracy for that type of prediction.

What the system does: Statistical models (logistic regression, gradient-boosted trees) trained on historical data generate predictions for six targets: payment probability, days to payment, churn risk, dispute probability, customer lifetime value, and DSO forecast. Each prediction is stored alongside its eventual outcome. The system tracks whether its 70% predictions actually come true 70% of the time. When accuracy drifts, the model is flagged for review.

3.4. Stage 4: Act

What happens: Based on the company state and predictions, the system proposes actions. “Send a soft reminder to Acme Corp about Invoice #4821. Use a personal tone because they have open support tickets and a renewal coming up. Here is the evidence for why this is the right action.”

What the user sees: An approval queue showing proposed actions with evidence bundles. Each proposal explains what it wants to do, why, what it considered, and what it expects to happen. The user approves or rejects with one click.

What the system does: Every proposed action passes through an 11-step pipeline before anything touches the real world. The system checks the agent’s authority, evaluates policy rules, validates parameters, checks rate limits, verifies budget, injects required disclosures (like “this message was composed with AI assistance”), estimates downstream effects, and decides whether to execute immediately, hold for human approval, or escalate.

Every executed action produces a complete evidence bundle: what was proposed, what facts were considered, what policies were applied, who approved it, what happened, and what the outcome was. These records are immutable and retained permanently.

3.5. Stage 5: Evaluate

What happens: Every action is graded on two dimensions. **Procedural:** did it follow the rules? Did it comply with policy, use the right tools, include required disclosures? **Outcome:** did it achieve the goal? Did the invoice get paid? Did the customer respond positively? Did anything unexpected happen?

What the user sees: Performance metrics per action type. “Collection emails: 94% procedural quality, 87% outcome quality, 0 incidents this month.”

What the system does: Execution traces are graded against the procedural checklist and outcome predictions. Results feed into the autonomy engine (Stage 6) and the calibration system (improving future predictions).

3.6. Stage 6: Learn

What happens: The system improves from its own operations. Predictions get more accurate as outcomes accumulate. Actions that consistently succeed earn the right to run with less human oversight. Actions that cause problems are immediately demoted to full human review.

What the user sees: An autonomy map showing which action types have earned which trust levels. “Soft collection reminders: autonomous. Payment plan offers: supervised. Dispute responses: locked.”

What the system does: The autonomy engine tracks trust per action type, per tenant. New actions start in supervised mode (every one needs human approval). After enough successful executions with high procedural and outcome scores, the system recommends promotion to automatic mode. A single serious incident triggers immediate demotion back to supervised. Trust is earned through evidence, never granted by default.

4. What Is Built Today

The system is live with Stripe as the first connected source. Here is what exists, what is in development, and what is planned.

Status	Details
LIVE	Stripe integration (webhooks, typed events, canonical objects). Event ledger with tamper-evident integrity. Object graph with customers, invoices, payments, disputes, and relationships. Prediction models for payment probability, churn risk, and dispute probability. Policy engine with deterministic rules. Action gateway with evidence bundles. Earned autonomy with promotion and demotion. Eight production dashboard views. 253 passing tests across unit, integration, contract, chaos, and load categories. Production infrastructure on Railway.
BUILDING	Additional data sources (Gmail, QuickBooks, CRM). Simulation engine for “what if” scenario analysis. Closed-loop calibration feeding outcomes back to improve predictions automatically.
TARGET	Cross-domain reasoning (AR + support + CRM in one model). Causal inference replacing statistical heuristics. Domain pack SDK for third-party extensions. Privacy-preserving learning across tenants.

4.1. The eight dashboard views

1. **Command Center:** Health indicators, activity stream, company metrics at a glance
2. **Company State:** Object graph explorer. Click any customer to see invoices, payments, communications, and relationships.
3. **Prediction Dashboard:** Payment probability scores, cash flow predictions, per-invoice risk assessment
4. **Autonomy Map:** Trust grid showing which agents can do what, and at which trust level
5. **Approval Queue:** Pending proposals with evidence bundles. One-click approve or reject.
6. **Policy Editor:** Write rules in natural language. The system enforces them as deterministic guards.
7. **Onboarding:** Stripe connection wizard. Live in 2 minutes.
8. **Landing Page:** Interactive walkthrough of the system’s value proposition

5. The World Model: How Predictions Work

The world model is the core intellectual property. It is how the system goes from raw events to actionable predictions. Here is what it does, mechanistically.

5.1. From events to objects

When Stripe sends a webhook ("Invoice #4821 payment failed"), the system does three things:

1. **Records the raw event** in the append-only ledger. This event is immutable and timestamped. It can never be changed or deleted.
2. **Updates the object graph.** Invoice #4821 is now marked as failed. The customer's payment history is updated. The relationship between the customer and the invoice reflects the new state.
3. **Triggers re-estimation.** The system recalculates predictions for the affected objects. Payment probability drops. Churn risk increases. The dispute probability may change based on the customer's history.

5.2. From objects to predictions

The system runs multiple prediction models, each focused on a specific target:

What it predicts	Output	Horizon
Will this invoice get paid?	Probability (0 to 1)	7, 30, 90 days
When will payment arrive?	Distribution of days	Per invoice
Will this customer leave?	Probability (0 to 1)	30, 90 days
Will this charge be disputed?	Probability (0 to 1)	Per invoice
What is this customer worth?	Distribution of dollars	12 months
What will DSO look like?	Distribution of days	30/60/90 days

Each prediction carries a confidence level and the model's historical accuracy for that prediction type. A model that predicts 70% payment probability should see approximately 70% of those invoices actually get paid. The system tracks this continuously and alerts when accuracy degrades.

5.3. What confidence actually means

When the system says "34% chance of payment in 7 days with confidence 0.82," it means two things:

- **34%** is the model's estimate of payment probability for this specific invoice, given everything it knows about the customer, the amount, the payment history, and the current state.
- **0.82 confidence** means the model has historically been accurate 82% of the time on similar predictions. If this number drops, the system flags the model for review and falls back to more conservative estimates.

This separation between "what do we think?" and "how good are we at thinking about this?" is what makes the predictions trustworthy. The system is honest about its own limitations.

5.4. The role of the AI model

The AI language model (GPT-4, Claude, Gemini, routed through a multi-provider system) is used for four specific functions:

1. **Reading unstructured data:** Parsing email bodies, extracting customer names from conversations, understanding the content of support tickets.
2. **Writing content:** Drafting collection emails, summarizing evidence bundles, generating human-readable explanations for predictions.
3. **Matching entities:** Recognizing that “Acme Corp” in Stripe and “ACME Corporation” in QuickBooks are the same customer.
4. **Explaining decisions:** Translating the system’s internal reasoning into natural language that a human operator can understand.

The AI model is **not** used for making predictions (that is the statistical model), making policy decisions (that is the rule engine), granting autonomy (that is the evidence-based promotion system), or storing data (that is the database).

Every AI call has a maximum cost limit, a retry limit, and a fallback template. If the AI is unavailable, the system continues operating. The AI makes things better. It does not make things possible.

6. Earned Autonomy: How Trust Works

The most important design decision in Nooterra is how it handles trust. Rather than giving AI agents a fixed set of permissions, the system lets each type of action independently earn the right to run with less human oversight.

6.1. The five trust levels

Level	What happens
Locked	This action type is forbidden. Cannot be used at all.
Shadow	The system proposes actions internally, but the user never sees them. Used for testing.
Supervised	The system proposes actions and the user must approve each one. This is the default for all new action types.
Auto + Review	The system executes the action automatically, and the user reviews afterward. Requires strong evidence of past success.
Autonomous	The system acts without human involvement. Requires extensive evidence and zero incidents.

6.2. How promotion works

An action type earns promotion through evidence. To move from Supervised to Auto + Review, the system needs at least 20 successful executions over 30 days, with high scores for both “did it follow the rules?” and “did it achieve the goal?” and almost no incidents.

To reach fully Autonomous, the bar is even higher: 50+ executions over 60 days, higher quality scores, and zero incidents of any kind.

6.3. How demotion works

Promotion is slow. Demotion is immediate. A single serious incident (a customer complaint, a policy violation, or an outcome that is significantly worse than predicted) triggers automatic demotion back to Supervised mode. The system notifies the human operator, shows the full evidence of what happened, and resets the trust score.

This asymmetry is deliberate. The cost of asking a human to approve an action they would have approved anyway is low. The cost of an unsupervised mistake is high.

6.4. Why this matters for adoption

Organizations will not deploy fully autonomous AI on day one. They should not. What they will deploy is a system that starts cautiously, proves its competence through observable evidence, and can be demoted instantly if it makes a mistake. This is how human employees earn trust. Nooterra applies the same model, backed by data.

7. Why Nooterra Wins

7.1. The competitive landscape

Category	Examples	What They Lack
AI Agent Frameworks	CrewAI, AutoGen, LangGraph	No world model. No governance. No persistent state. Developer tooling, not a product.
Vertical AR Tools	Tesorio, Upflow, Bill.com	Timer-based reminders. No state estimation, no cross-domain awareness, no earned autonomy.
Workflow Automation	Zapier, Make, n8n	Task execution without understanding consequences. No simulation, no governance.
Foundation Model Cos.	Anthropic, OpenAI, Google	Horizontal platforms selling APIs. Vertical domain runtimes require data they do not have.

7.2. What about the big AI companies?

The most common question: will Anthropic, OpenAI, or Google build this?

They could build a general-purpose state-tracking layer. The value in Nooterra is not the state-tracking layer. It is the calibrated prediction models trained on proprietary action-outcome data, the domain-specific policy engines, and the tenant-level trust histories that accumulate over months of real use. These are vertical data assets, not features.

Foundation model companies build horizontal platforms. Nooterra builds the vertical intelligence layer on top. We use their APIs. We are not competing for the same budget.

7.3. The four moats

Proprietary data. Every tenant's approvals, rejections, outcomes, and incidents become training signal. This data does not exist in any public dataset. No one else has enterprise action-outcome pairs with full context, policy state, and confidence intervals attached.

Trust lock-in. Once a company has spent months training autonomy levels, building custom policies, and calibrating predictions, switching means starting at zero trust. Trust histories cannot be exported.

Engineering depth. The combination of event ledger, object graph, state estimator, policy engine, action gateway, and autonomy engine represents 12 to 18 months of infrastructure engineering. The first team to ship it with a working product has a structural head start.

Cross-domain compounding. Each new data source increases the value of every existing source. When AR, support, and CRM are all connected, the system sees patterns no single-domain tool

can see: “Do not send that collection email because this customer has an open support ticket and a pending renewal.”

8. Commercial Model

Nooterra prices on business value, not compute. Customers do not care how many AI tokens were consumed. They care whether cash arrived faster and whether bad actions were prevented.

Tier	Price	For	What You Get
Sandbox	$0	Evaluation	1 Stripe account, shadow mode, time-boxed trial
Starter	$149/mo	Founder-led SMBs	1 agent, Stripe world model, approval queue, predictions
Business	$499/mo	Growing businesses	Full autonomy, multiple runtimes, Slack approvals
Finance Ops	$799–1,500/mo	Finance teams	Multi-user, audit/export, custom policies, premium support
Enterprise	Custom	Mid-market+	Multi-system, SSO/SCIM, custom connectors, SLA

8.1. Unit economics

AI cost per customer: approximately $50/month. At the Business tier ($499/month), that is an 87% gross margin. Even at Starter ($149/month), the margin is approximately 56%. These economics improve as models get cheaper and the system optimizes which AI model to use for which task (routing simple tasks to cheaper models, complex judgment to expensive ones).

8.2. Go-to-market

Months 1–6: Design partner pilots. Hand-selected B2B service companies. Free access in exchange for weekly feedback. Goal: 5–10 partners, measurable reduction in days sales outstanding.

Months 4–8: Invite-only launch. 50 companies from the waitlist. Paid Starter and Business tiers. Content marketing with partner case studies.

Months 7–12: Open access. Self-serve sandbox trial. Bookkeeper and accountant referral program.

9. The Vision: From Wedge to Platform

The product starts narrow. That is deliberate.

Stripe-first accounts receivable is chosen because it has the cleanest event stream, the fastest feedback loops, the most measurable outcomes, and the simplest onboarding. It is the fastest path to a validated world model and a meaningful dataset of action-outcome pairs.

The expansion sequence builds on this foundation:

Phase	Timeline	What Gets Built	How We Know It Worked
AR Wedge	Months 1–6	Stripe world model. Shadow collections. Governed actions. Earned autonomy.	5–10 design partners. Measurable DSO reduction.
Finance Control Plane	Months 7–12	Disputes, refunds, cash forecasting. Gmail and QuickBooks integration. Scenario simulation.	200 paying customers. First case studies published.
Multi-Domain Platform	Months 13–24	CRM integration. Cross-domain relationships. Domain pack architecture. Support ops.	1,000+ customers. Enterprise deals. Platform thesis validated.

Each phase depends on the previous one. Expansion into disputes requires the collections infrastructure. Multi-source reasoning requires the single-source model to be calibrated and trusted. Domain packs require the platform architecture to be proven.

Expansion is earned, not declared.

> **The long-term vision:** Any business can provision a governed AI operator the way it provisions a database. Identity, permissions, budget, memory, world model access, and a liability envelope, all in one step. The system learns from every action it takes, earns trust through evidence, and gets demoted the moment it makes a mistake.

N O O T E R R A

Enterprise World Model Platform
nooterra.ai

April 2026. Confidential.